

SEGURIDAD — EN REDES —

TEMPORADA 3

SENSORES EN LA BRECHA - PREVENCIÓN Y DETECCIÓN DE INTRUSOS



GRADO EN INGENIERÍA DE LA CIBERSEGURIDAD
DIRECTED BY CÉSAR CÁCERES TALADRIZ

INTRODUCCIÓN – TEMPORADA 3

La calma lograda tras la segmentación del Hawkins National Laboratory no duró mucho. Las Zonas Funcionales (VLANs) permanecen firmes, los enlaces troncales controlan su tráfico y los puntos críticos están protegidos... pero los científicos vuelven a detectar señales inquietantes. Son *ecos*. Pequeñas vibraciones digitales que no deberían existir: paquetes que reaparecen donde no fueron enviados, rastros repetidos, patrones demasiado precisos como para ser fruto del azar. El Upside Down ha aprendido a moverse con más sutileza. No intenta atravesar los muros: intenta *imitarlos*. El Escuadrón Hawkins comprende que la contención ya no basta. Si el enemigo empieza a moverse dentro, aunque sea como un susurro entre paquetes, es imprescindible desplegar sensores: sistemas capaces de observar, identificar, correlacionar y actuar antes de que la anomalía se convierta en una intrusión real. Aquí comienza la **Temporada 3: Sensores en la brecha**.

Recordad que podéis consultar más información en el **Tema 4 sobre Prevención y detección de intrusos: IPS e IDS**, y que de este tema tendréis que superar dos pruebas, una teórica y una práctica. Si realizáis un informe de las pruebas de laboratorio no es necesario entregarlo, aunque es deseable que registréis lo que hacéis para poder recordarlo.

Esta temporada tiene tres episodios en los que aprenderás a configurar y activar **IPS/IDS**, en el primero con *Cisco IOS IPS*, en el segundo con **Snort** y en el tercero con **Suricata**, estos últimos son de los sistemas más extendidos en la industria. Instalarás, ajustarás y probarás estas herramientas como si fuesen auténticos sensores del laboratorio, desplegados para escuchar cualquier movimiento extraño en la red.

El objetivo no es solo aprender a configurar estas herramientas, sino entender cómo “escuchan” la red, cómo identifican señales improbables y cómo te permiten tomar el control cuando algo intenta cruzar la brecha entre mundos.

Los **eventos anómalos** que encontréis durante el camino, como este que tenéis a la derecha a modo de ejemplo, reflejan momentos de tensión en los que será necesario validar que el Escuadrón mantiene el control de la situación. Cada vez que resolváis un evento anómalo debéis **comunicarlo mediante Wooclap** (una vez **todos los miembros del equipo lo tengáis resuelto** y claro).



PREPARACIÓN DEL HAWKINS NATIONAL LABORATORY

Como en anteriores ocasiones, necesitaremos el Cisco Packet Tracer. Para su descarga debes acceder a la plataforma Cisco Networking Academy (www.netacad.com) y con los datos de usuario que te facilitamos al darte de alta podrás descargar el Cisco Packet Tracer (aquí tienes el enlace directo al [Packet Tracer](#)). Ten en cuenta que debes activar la instalación con tu usuario y clave la primera vez que lo ejecutes. Si nunca has utilizado el Packet Tracer, te recomendamos que hagas el Curso de introducción que podrás encontrar en [Introduction to Packet Tracer](#), al que te puedes autoinscribir.

Para el segundo y tercer episodio necesitarás el IPS de código abierto Snort (www.snort.org) y Suricata (suricata.io), que deberás instalar en la máquina que quieras defender. Utilizaremos un esquema de dos máquinas virtuales (una atacante y otra víctima) que deberán compartir red. La máquina atacante podría ser un Kali y la víctima un Ubuntu (por ejemplo, la máquina de SeedLabs).

Episodio 1 – El eco sospechoso (1)

Desde que las VLAN aislaron el laboratorio, los movimientos del enemigo se han vuelto más sutiles. Ya no se manifiesta como un ataque directo, sino como pequeños ecos: pings fuera de contexto, patrones repetidos, firmas

que no deberían aparecer. El equipo decide **desplegar un IPS interno, integrado en el router Cisco**, para analizar el tráfico y alertar de cualquier anomalía.

El IOS IPS actúa como un sensor avanzado: examina cada paquete, lo compara con su base de firmas y decide si debe permitirlo, registrarlo o bloquearlo. En este episodio, activarás este sensor, lo posicionarás estratégicamente en la red y ajustarás su comportamiento. Aunque IOS IPS no se emplea desde 2018 en routers Cisco, veremos este sistema al poderse simular de manera sencilla con Cisco Packet Tracer.

El Escuadrón Hawkins convertirá, paso a paso, un router aparentemente ordinario en un dispositivo capaz de escuchar y responder a señales que el laboratorio no puede permitirse ignorar.

En este episodio, activarás el primer sistema de vigilancia continuo del Hawkins Lab.

Nota: Los comandos que se sugieren en la práctica son los comandos Cisco completos, no abreviados, para facilitar la comprensión. Puedes emplear estos o cualquier otro que pienses haga lo mismo. Y recuerda que funciona el tabulador para completar comandos y el ? para ayuda.

1. Configuración de IOS IPS usando CLI (CCNA Security 5.4.1.2)

TOPOLOGÍA

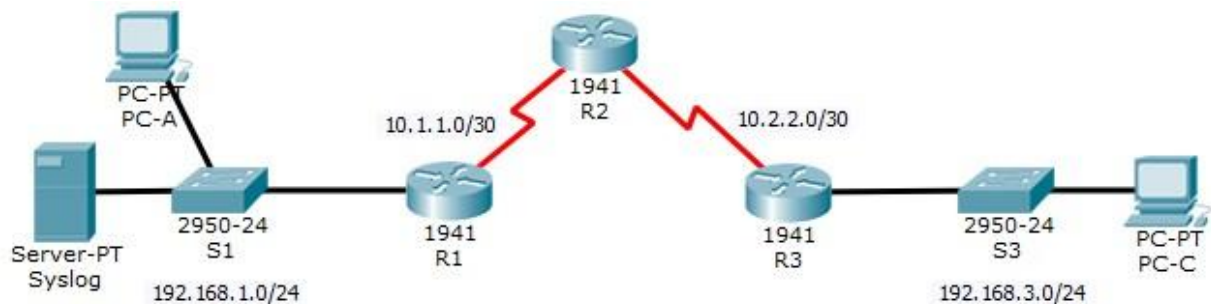


TABLA DE DIRECCIONAMIENTO

Device	Interface	IP Address	Subnet Mask	Default Gateway	Switch Port
R1	G0/1	192.168.1.1	255.255.255.0	N/A	S1 F0/1
	S0/0/0	10.1.1.1	255.255.255.252	N/A	N/A
R2	S0/0/0 (DCE)	10.1.1.2	255.255.255.252	N/A	N/A
	S0/0/1 (DCE)	10.2.2.2	255.255.255.252	N/A	N/A
R3	G0/1	192.168.3.1	255.255.255.0	N/A	S3 F0/1
	S0/0/0	10.2.2.1	255.255.255.252	N/A	N/A
Syslog	NIC	192.168.1.50	255.255.255.0	192.168.1.1	S1 F0/2
PC-A	NIC	192.168.1.2	255.255.255.0	192.168.1.1	S1 F0/3
PC-C	NIC	192.168.3.2	255.255.255.0	192.168.3.1	S3 F0/2

OBJETIVO

Este apartado tiene dos partes. En la primera configuramos el IPS y creamos una regla que aplicamos en una de las interfaces del router R1. En la segunda parte comprobamos que esa regla está en funcionamiento en el router, modificándola para que impida realizar pings hacia la red interna y registrando los eventos en un servidor Syslog.

El servidor y los PCs han sido configurados y los routers tienen las siguientes contraseñas establecidas:

- Clave de consola: ciscoconpa55
- Clave de enable: ciscoenpa55
- Usuario y clave de SSH: SSHadmin / ciscosshpa55

En el Cisco Packet Tracer los routers ya tienen los ficheros de firmas instalados en la flash y por eso no es necesario configurar la clave pública y hacer la importación manual de dichos ficheros.

1.1. HABILITA EL IOS IPS

Paso 1: Habilita el paquete de Security Technology

- a) En R1, ejecuta el comando `show version` para ver la información de licencias del paquete de tecnologías.
- b) Si el paquete Security Technology no está habilitado, ejecuta el siguiente comando para habilitarlo.
`R1(config)# license boot module c1900 technology-package securityk9`
- c) Acepta el acuerdo de licencia de Seguridad.
- d) Guarda el running-config (cópialo al startup-config).
- e) Haz un reload del router para habilitar la licencia.
- f) Comprueba que el paquete de Seguridad está habilitado usando el comando `show version`.

Paso 2: Verifica la conectividad entre PCs

- a) Haz un ping de PC-C a PC-A (debería funcionar).
- b) Haz un ping de PC-A a PC-C (debería funcionar).

Paso 3: Crea una carpeta para la configuración del IOS IPS en la flash

En R1 crea un directorio en la flash usando el comando `mkdir`. Ponle de nombre `ipsdir`.

Paso 4: Configura la ubicación del almacén de firmas IPS

En R1 configura que esa carpeta que acabas de crear sea la ubicación del almacén de firmas del IPS.

Paso 5: Crea una regla IPS

En R1 crea una regla IPS poniendo de nombre `iosips` utilizando el comando `ip ips name <nombre>` en el modo de configuración global.

Paso 6: Habilita el logging

IOS IPS soporta el uso de syslog para enviar notificaciones de eventos (habilitadas por defecto). Si la consola de logging está habilitada se mostrarán los mensajes de IPS syslog.

- a) Habilita el syslog si no lo está.
- b) Si es necesario, emplea el comando `clock set` desde el modo de ejecución privilegiada para configurar la hora y fecha en el reloj.

- c) Verifica que el servicio de timestamp para el logging está habilitado en el router empleando el comando `show run`. Habilita el servicio timestamp si no está habilitado.
- d) Configura para que se envíen los mensajes de log al servidor syslog con dirección IP 192.168.1.50.

Paso 7: Configura el IOS IPS para que emplee las categorías de firmas

Retira la categoría `all` con el comando `retired true` (todas las firmas que vienen con el paquete de firmas). Habilita la categoría `IOS_IPS Basic` con el comando `retired false`.

Paso 8: Aplica la regla IPS a una interfaz

Aplica la regla IPS a una interfaz con el comando `ip ips <nombre> <dirección>` en el modo de configuración de interfaz. Aplica la regla a la salida de la interfaz G0/1 de R1. Tras habilitar el IPS, algunos mensajes de log se enviarán a la consola indicando que los motores IPS están siendo inicializados. También se enviarán estos mensajes de log al servidor de Syslog (abre su pestaña *Services* y comprueba cómo han aparecido estos mensajes).

1.2. VISUALIZA LAS FIRMAS

Paso 1: Comprueba las firmas habilitadas

Con el comando `show ip ips configuration` o el comando `show ip ips all` se puede ver el resumen del estado de la configuración del IPS. ¿A qué interfaces y en qué dirección se está aplicando la regla `iosips`?

Paso 2: Visualiza la firma que detecta pings

Como se puede observar, únicamente tiene una firma activa, que se trata de la firma que detecta tráfico de solicitud de echo (signature 2004, subid 0). Emplea el comando `show ip ips signature sigid XXXX subid XXXX` para visualizar la información concreta de esta firma. ¿Está habilitada? ¿Está compilada? ¿Qué acción realiza esta firma?

1.3. MODIFICA LA FIRMA

Paso 1: Cambia el event-action de la firma

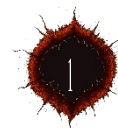
“Des-retira” la firma de solicitud de echo (signature 2004, subid ID 0), habilítala, y cambia la acción de la firma a alertar y desechar. Ve visualizando los cambios en la firma según los vas haciendo.

Paso 2: Verifica que el IPS está funcionando correctamente

- a) Haz un ping de PC-C a PC-A. ¿Puedes? ¿Deberías poder?
- b) Haz un ping de PC-A a PC-C. ¿Puedes? ¿Deberías poder?

Paso 3: Observa los mensajes de syslog

- a) Haz click en el servidor Syslog.
- b) Selecciona la pestaña Services.
- c) En el menú de navegación de la izquierda, selecciona SYSLOG para ver el fichero de logs.



Episodio 2 – El sensor que despierta (2.A, 2.B y 2.C)

Aunque el IPS de Cisco proporciona una primera línea de detección, el equipo pronto descubre que necesita algo más profundo. Algo más flexible, capaz de adaptarse, detectar patrones concretos y registrar actividad en detalle.

Es entonces cuando aparece **Snort**: un sensor híbrido, ligero y poderoso, utilizado en miles de sistemas reales. Para el Hawkins Lab, Snort se convertirá en el oído más afinado dentro de su red.

En este segundo episodio, el Escuadrón Hawkins aprenderá a instalar y hacer la configuración básica de uno de los IPS más empleados, el IPS de código abierto Snort (www.snort.org). Con cada regla nueva, el laboratorio gana un sensor más detallado, capaz de identificar ecos específicos entre miles de paquetes legítimos. Para esta práctica vamos a utilizar dos máquinas virtuales, la de la víctima que vamos a proteger y que podrá ser cualquier Linux que tengas instalado (por ejemplo una máquina SEED donde instalaremos Snort), y para la atacante utilizaremos por ejemplo un Kali Linux o un Parrot OS. Snort puede ejecutarse como root o con el usuario snort. En entornos de producción siempre lo ejecutaremos con su propio usuario (snort), pero en esta práctica lo haremos como root para evitar problemas de permisos. Tienes información y descargas de Snort en su web (<https://www.snort.org/downloads>) y un Manual (<https://www.snort.org/documents/snort-3-1-18-0-onubuntu-18-20>), aunque también hay múltiples tutoriales en Internet.

Es la primera vez que el Escuadrón no solo mira *qué* pasa en la red, sino *cómo* pasa.

2.1. Introducción a Snort 3

Snort 3 es la evolución moderna del motor de detección de intrusiones Snort 2. A diferencia de la versión clásica, Snort 3 incorpora una arquitectura modular, soporte multihilo real, un sistema de configuración basado en Lua y un pipeline de inspección optimizado. En esta práctica veremos su función como IDS únicamente, no como IPS.

2.1.1. INSTALACIÓN Y CONFIGURACIÓN DE SNORT 3

Primero instalaremos Snort en su versión 3 en la máquina víctima para protegerla, empleando el script facilitado en clase o siguiendo documentación o tutoriales de Internet. Una vez instalado comprobamos que todo ha ido bien con el comando `snort -v`. El fichero de configuración de Snort 3 se puede encontrar en `/usr/local/etc/snort/snort.lua`, y viene bastante documentado y se entiende fácilmente. En este fichero se define el pipeline, módulos, decoders y configuraciones internas del motor, y se apoya en las definiciones que tiene por defecto en otro fichero, el `/usr/local/etc/snort/snort_defaults.lua`.

2.1.2. ESTRUCTURA DE REGLAS Y FICHEROS .RULES

Las reglas en Snort se almacenan en ficheros `.rules`, y en la práctica únicamente trabajaremos con el conjunto de reglas locales que introduciremos en el fichero `/etc/snort/rules/local.rules`. Las reglas en Snort tienen la siguiente estructura:

```
<Rule Action> <Protocol> <Source IP Address> <Source Port> <Direction Operator>  
<Destination IP Address> <Destination Port> (rule options: message, identification  
number, revision number)
```

1. Rule action: Define lo que Snort tiene que hacer con el paquete. Tiene ocho opciones (las cinco primeras como IDS y las otras tres como IPS, que requieren modo *inline*):

- a. alert: genera una alerta y hace un log del paquete.

- b. log: hace un log del paquete.
 - c. pass: ignora el paquete.
 - d. activate: alerta y active una regla dinámica.
 - e. dynamic: se mantiene inactiva hasta que se activa con la regla activate, entonces actúa como log.
 - f. drop: bloquea y hace log del paquete (modo *inline* requerido).
 - g. reject: bloquea el paquete, hace log del mismo, y envía un TCP reset si el protocolo es TCP o un mensaje ICMP de “port unreachable” si es UDP (modo *inline* requerido).
 - h. sdrops: bloquea el paquete pero no hace log del mismo (modo *inline* requerido).
2. Protocol: Indica el tipo de protocolo a analizar (icmp, udp, tcp o ip).
3. Source IP Address: la dirección IP de origen del paquete. Puede ser una ip, una subred, una variable del fichero de configuración (como \$HOME_NET), “any”, una lista (por ejemplo, [192.168.1.0/24,10.0.1.0/16])...
4. Source Port: El número de Puerto del que proviene el paquete. Puede ser un número, una variable, “any”, o un rango (1:1024).
5. Direction operator: Puede ser -> para indicar que debe comprobar el tráfico de origen a destino, o <> para que lo compruebe en ambos sentidos.
6. Destination IP Address: la dirección IP destino.
7. Destination Port: El número de Puerto destino.
8. Rule options: Hay muchas opciones, agrupadas en cuatro tipos (general, payload, non-payload y postdetection). Las más empleadas son:
- a. msg: el mensaje que se muestra en el fichero de log.
 - b. sid: el id de la regla. No puede haber dos reglas con el mismo id, y como las primeras 1,000,000 están reservadas, para las locales se deberá utilizar un sid mayor de un millón.
 - c. rev: el número de revisión de la regla, para refinar y actualizarla. Si coinciden dos reglas con el mismo sid, Snort ejecutará la primera que encuentre, independientemente del valor de rev, siendo este meramente informativo.
 - d. content: permite buscar un patrón específico dentro de los datos inspeccionados por Snort. Puede utilizarse junto a modificadores como *nocase* (búsqueda sin distinguir mayúsculas/minúsculas), *depth* (límite de bytes donde buscar), u *offset* (desplazamiento del inicio de búsqueda). Además, en protocolos de aplicación, como HTTP, Snort 3 requiere especificar el buffer donde debe realizar la inspección, para indicar en qué parte del mensaje se debe buscar el contenido. Algunos buffers habituales son: *http_uri* (examina la URI de la petición), *http_header* (analiza las cabeceras HTTP), *http_client_body* (busca en el cuerpo enviado por el cliente), *http_cookie* (busca dentro de la cookie), *file_data* (busca en los datos reconstruidos de archivos). Por tanto, una regla con content en tráfico HTTP suele incluir primero el buffer adecuado, seguido del patrón que se desea localizar.
 - e. pcre: permite utilizar expresiones regulares avanzadas basadas en la librería PCRE2. Esta opción resulta útil para detectar patrones complejos en el tráfico, más allá de lo que permite una simple búsqueda con content (ver www.pcre.org o <https://perldoc.perl.org/perire>).
 - f. ttl: comprueba el valor del tiempo de vida del paquete.
 - g. flags: comprueba las flags de TCP como SYN, ACK, RST o FIN.

- h. flow: permite aplicar la regla a un determinado sentido de la comunicación, especificando *from_client*, *to_client*, *to_server*, *from_server*, o si está establecida o no la conexión TCP, con *established* y *not_established*.

2.1.3. EL COMANDO SNORT

El comando snort tiene muchas opciones, estas son las principales que usaremos:

- c <configfile>: especifica el fichero de configuración, normalmente `/usr/local/etc/snort/snort.lua`.
- R <rulesfile>: carga reglas adicionales desde el archivo `.rules`, normalmente `/etc/snort/rules/local.rules`.
- A <modo>: selecciona el formato de salida de la alerta (*alert_fast*, compacto y legible, *alert_full*, información detallada, *alert_json*, salida en JSON, *none*, desactiva las alertas).
- i <interface>: especifica la interfaz que debe escuchar Snort.
- v: la opción verbose muestra más información por alerta.
- d: muestra parte del payload en las alertas.
- e: muestra información de las cabeceras de la capa de enlace.
- k none: deshabilita la verificación interna que hace Snort del checksum.

2.2. Detectando actividad del UpsideDown: ICMP Echo Request

En el Laboratorio de Hawkins hemos observado un patrón preocupante: algunas entidades del UpsideDown utilizan tramas ICMP Echo Request (los “pings” de toda la vida) para comprobar si ciertos equipos siguen activos o vulnerables. Estas señales suelen preceder a ataques más complejos, por lo que nuestro IDS debe ser capaz de detectarlas de inmediato. En este apartado configuraremos Snort para alertarnos cada vez que desde la máquina víctima, que estamos vigilando, salga una solicitud ICMP tipo 8 (Echo Request). Es el equivalente a detectar “ecos” anómalos provenientes del UpsideDown.

Vamos por tanto a configurar Snort para detectar si alguien intenta hacer ping desde la máquina víctima que queremos proteger. Tendrás que insertar la siguiente regla en el fichero `local.rules` (poniendo la ip de la máquina víctima donde corresponda):

```
alert icmp any any -> any any (msg:"Ping detected"; itype:8; sid:1000477; rev:1)
```

Siguiendo lo indicado en la sección previa, vemos que esta regla creará una alerta con cada paquete ICMP de cualquier puerto a cualquier puerto de cualquier dirección. Pero no de cualquier paquete ICMP, sino únicamente los de tipo 8, que son las solicitudes de echo, que es lo que queremos detectar (las respuestas serían tipo 0).

Ejecutamos ahora Snort en la máquina víctima:

```
snort -d -e -v -c /usr/local/etc/snort/snort.lua -R /etc/snort/rules/local.rules -A alert_fast -i <interfaz> -k none
```

Para comprobar que funciona, haz ping durante un par de segundos desde la máquina víctima y comprueba que Snort captura únicamente las alertas (ICMP Echo request desde la máquina víctima). Prueba también a hacerle ping a la máquina víctima desde la máquina atacante para comprobar que de ese tráfico no se alerta. Te saldrá algo similar a la figura siguiente. Analiza la información que proporciona.

Para que las alertas se almacenen en un fichero de logs en lugar de salir por consola, muy útil sobre todo cuando tienes muchas o cuando quieres enviar ese fichero a analizar por otros programas, puedes emplear la opción `-A alert_fast --lua "alert_fast = { file = true }"` (o escribir esta opción directamente en su apartado del fichero `snort.lua`). Esto creará un fichero `alert_fast.txt` en la carpeta donde ejecutes snort.

```
03/15-08:47:32.605581 [**] [1:1000477:1] "Ping detected" [**] [Priority: 0] {ICMP} 10.0.2.12 -> 10.0.2.13
08:00:27:4B:E0:66 -> 08:00:27:AC:64:0E type:0x800 len:0x62
10.0.2.12 -> 10.0.2.13 ICMP TTL:64 TOS:0x0 ID:8499 IpLen:20 DgmLen:84 DF
Type:8 Code:0 ID:12 Seq:2 ECHO

snort.raw[56]:
- - - - -
64 AA B6 69 00 00 00 00 8D 3C 09 00 00 00 00 00 d..i....<.....
10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F .....
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F !"#%&'()*+,-./
30 31 32 33 34 35 36 37 01234567
- - - - -

03/15-08:47:33.607609 [**] [1:1000477:1] "Ping detected" [**] [Priority: 0] {ICMP} 10.0.2.12 -> 10.0.2.13
08:00:27:4B:E0:66 -> 08:00:27:AC:64:0E type:0x800 len:0x62
10.0.2.12 -> 10.0.2.13 ICMP TTL:64 TOS:0x0 ID:8721 IpLen:20 DgmLen:84 DF
Type:8 Code:0 ID:12 Seq:3 ECHO

snort.raw[56]:
- - - - -
65 AA B6 69 00 00 00 00 5F 45 09 00 00 00 00 00 e..i...._E.....
10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F .....
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F !"#%&'()*+,-./
30 31 32 33 34 35 36 37 01234567
- - - - -
```

¿Cómo harías para alertar si hacen ping “a” la máquina víctima, en lugar de “desde” la máquina víctima? ¿Y para hacerlo en ambos sentidos? Prueba ambas posibilidades.

¿Y cómo harías para además de alertar, denegar los ping que vayan a tu máquina víctima? ¿funciona?



2.3. Configurar Snort para alertar si se navega a sitios prohibidos

Los científicos de Hawkins han descubierto que ciertas criaturas del UpsideDown tratan de acceder a portales de conocimiento humano para aprender más sobre nuestro mundo. Entre ellos, uno de los portales más utilizados es www.wikipedia.org. Según los registros de Eleven y los análisis de Mr. Clarke, acceder a ese dominio durante un ataque puede indicar que una máquina ha sido comprometida, o que el Mind Flayer está recopilando información, o que una entidad está intentando “aprender” sobre nuestro mundo. Tu objetivo en este apartado será configurar Snort para **alertar si desde la máquina víctima se intenta acceder a este dominio prohibido**.

La primera forma de hacerlo podría ser comprobando directamente en la regla si la cadena “www.wikipedia.org” está en el paquete inspeccionado. La regla debería por tanto generar una alerta cuando un paquete TCP saliente de cualquier puerto de \$HOME_NET a los \$HTTP_PORTS de \$EXTERNAL_NET contenga “www.wikipedia.org”:

```
alert tcp <IP_Snort> any -> $EXTERNAL_NET $HTTP_PORTS (msg:"Detectada solicitud con
www.wikipedia.org"; content:"www.wikipedia.org"; sid:1000004; rev:1;)
```

Una vez guardada la regla en `local.rules`, iniciamos Snort y visitamos la web www.wikipedia.org para comprobar que efectivamente se está registrando en pantalla dicha actividad por Snort. Tened muy en cuenta por qué interfaz salimos de esa máquina a Internet, pues será la que tenga que poner en el comando Snort. Además, si utilizamos un navegador es probable que funcione unas veces sí y otras no, según si la petición está cacheada o no, por lo que para las pruebas en este apartado es más sencillo hacer peticiones web desde un terminal con el comando `curl -v www.wikipedia.org` que desde un navegador.

Aunque efectivamente esa regla funciona y registra si intento navegar a ese sitio web, también generará muchos falsos positivos, pues cualquier tráfico que contenga esa cadena (sea o no el sitio web) hará saltar la alerta.

Esto nos lleva a una segunda forma de hacerlo, comparando dentro de la cabecera HTTP de la solicitud GET. Para ello utilizaremos los buffers de HTTP disponibles en Snort 3, de forma que puedo especificar que me alerte de los GET con `http_method; content:"GET"`, en los que el Host cumpla cierta expresión regular con `http_header; pcre:"<expresión_regular>"`.

Visita www.wikipedia.org y observa el formato de la solicitud GET realizada (con Burp, con el inspector de cabeceras del navegador, o incluso con el propio Snort). Ahora te basarás en ese formato para crear tu regla, que será más específica, alertando únicamente cuando se trate de un GET que vaya a ese sitio web. Para ello miraremos primero que tenga el método GET, y después "Host: [algo] wikipedia.org" con case-insensitive. Ese [algo] será `www.` o `http://` o nada... Para construir esa regla utilizaremos expresiones regulares (ver www.pcre.org o <http://perldoc.perl.org/perlre>).

En lugar de sobrescribir la regla anterior podemos comentarla. También podrías utilizar las versiones de reglas de snort y añadir una regla con el mismo sid, pero la siguiente revisión, pero es importante que en este caso tengas en cuenta que se ejecutará la que esté primero en el fichero.

```
alert http <IP_Snort> any -> $EXTERNAL_NET $HTTP_PORTS (msg:"Detectada solicitud a Wikipedia"; http_method; content:"GET"; http_header; pcre:"/^Host\s*:\s*([a-z0-9-]+\.)*wikipedia\.org\s*$"/mi"; sid:1000006; rev:1;)
```

La expresión regular empleada en este caso se detalla en esta tabla:

Fragmento	Significado
<code>^</code>	Ancla de inicio de línea (con flag m, actúa línea a línea)
<code>Host</code>	Literal, el nombre de la cabecera
<code>\s*</code>	0 o más espacios tras Host
<code>:\s*</code>	: literal seguido de 0 o más espacios
<code>([a-z0-9-]+\.)*</code>	Grupo de subdominio (letras, dígitos o guiones + punto literal), repetido 0 o más veces
<code>wikipedia</code>	Literal
<code>\.</code>	Punto literal (escapado para que no sea "cualquier carácter")
<code>org</code>	Literal
<code>\s*</code>	0 o más espacios tras org
<code>\$</code>	Ancla de fin de línea (con flag m, actúa línea a línea)
<code>mi</code>	Flags: m multilinea, i no importa si mayúscula o minúscula (case insensitive)

Una vez guardada la regla en `local.rules`, iniciamos Snort y solicitamos con curl la web www.wikipedia.org para comprobar que efectivamente se está alertando por pantalla de dicha actividad.



2.4. Detectando ataques de reconocimiento desde el UpsideDown

En este apartado vamos a ponernos en la piel del Mind Flayer y sus criaturas. Supongamos que han conseguido un pequeño punto de apoyo dentro de Hawkins (nuestra máquina atacante Kali) y ahora intentan enumerar información, buscar rutas escondidas, probar métodos HTTP extraños y descubrir posibles vulnerabilidades.

Nuestro objetivo será, por tanto, lanzar diferentes ataques desde la Kali para después observar qué alertas generan las reglas de la comunidad de Snort 3 en la máquina víctima. Además, podremos analizar las reglas reales que se activan y comprender cómo interpretan el tráfico malicioso. Al revisar las firmas, adoptarás el papel de científico de Hawkins, analizando cómo el Laboratorio detecta patrones reales empleados por el UpsideDown para explorar y preparar ataques.

Lo primero será descargarnos las reglas de la comunidad Snort (<https://www.snort.org/downloads#rules>) y copiarlas en una carpeta de nuestra máquina SEED (víctima) para poder llamarlas desde el comando snort (en lugar de llamar a las local.rules).

Para que el ataque sea viable, aseguraremos que la máquina víctima tiene un servidor web activo con `sudo python3 -m http.server 80` (o instalando un servidor Apache). Este portal web actuará como una puerta de Hawkins, que será explorada desde el UpsideDown.

Y también en la máquina víctima lanzaremos nuestro IDS Snort con las reglas de comunidad activas, de una forma similar a esta:

```
sudo snort -c /usr/local/etc/snort/snort.lua -R /etc/snort/rules/community.rules -i  
enp0s3 -A alert_fast -k none -d -e -v
```

Vamos a probar ahora 4 herramientas de ataque de reconocimiento desde el Upside Down (además de todas las que quieras probar tú extra). Cada una tendrá una forma distinta de explorar o presionar nuestro portal web. Esto generará diferentes patrones de tráfico, permitiendo ver varios tipos de alertas HTTP reales en Snort 3.

A) Petición web directa a una ruta prohibida

Este primer ataque representa a una criatura del Upside Down probando puertas prohibidas del portal, rutas que ningún usuario humano debería solicitar. Lo que hace realmente es probar con el comando curl si se tiene acceso a alguna ruta “interesante” en el servidor web.

Ejecuta desde la máquina atacante el comando `curl -v http://<ip_server>/etc/passwd` y observa la alerta generada en Snort (por consola o en fichero). Busca ahora esa regla en el fichero `community.rules` que has cargado y analízala en detalle. Estudia cómo se ha escrito la regla, qué patrones busca, en qué buffers http, qué heurísticas usa para diferenciar el tráfico maligno del resto, etc. Para simplificar la búsqueda, fíjate en el identificador de la regla (sid), que en este caso es el 1122, por lo que podrías hacer la búsqueda con el comando `grep -R "sid:1122" /etc/snort/rules/` si está en esa carpeta el fichero `community.rules`.

B) Enumeración de rutas conocidas

El Mind Flayer intenta analizar con nmap las defensas del portal, enviando señales en diferentes métodos (TRACE, OPTIONS, HEAD) para ver cómo responde el sistema. El atacante trata de enumerar rutas conocidas (con http-enum) y conocer los métodos HTTP aceptados por el servidor web víctima (con http-methods). De esta forma intenta revelar información sensible sobre la configuración del servidor.

Ejecuta el comando `sudo nmap -sV --script="http-enum,http-methods" -p 80 <victima>` y observa las alertas generadas por Snort. Busca igual que antes algunas de las reglas en el fichero `community.rules` y analízalas.

C) Enumeración agresiva de rutas ocultas

La criatura del UpsideDown ahora va a emplear la herramienta Gobuster para explorar metódicamente cada habitación del portal, probando nombres de puertas ocultas para ver si alguna conduce a una Brecha. Este programa (similar a `dirb`) realmente hace un bruteforce de directorios y archivos usando una wordlist, enviando decenas o cientos de solicitudes HTTP distintas. Si no tienes instalado gobuster en tu Kali, puedes hacerlo con `sudo apt install gobuster -y`.

Una vez instalado, ejecuta el comando `sudo gobuster dir -u http://<victima>/ -w /usr/share/wordlists/dirb/common.txt` y observa las alertas generadas por Snort. Busca igual que antes algunas de las reglas en el fichero `community.rules` y analízalas.

D) Fuzzing de rutas y parámetros

Nos enfrentamos ahora a una criatura más sofisticada del UpsideDown, que con `wfuzz` va probando múltiples combinaciones para encontrar grietas o inconsistencias en el portal. `wfuzz` es similar a Gobuster, pero permite realizar fuzzing más flexible sobre rutas, parámetros o incluso cabeceras.

Como en los anteriores casos, ejecuta el siguiente comando en la máquina atacante `wfuzz -c -z file, /usr/share/wordlists/dirb/common.txt http://<victima>/FUZZ` y observa las alertas generadas por Snort. Busca igual que antes algunas de las reglas en el fichero `community.rules` y analízalas.

Para terminar el episodio, puedes probar más herramientas de reconocimiento como Nikto (en su modo ligero con `-Tuning 1`), Dirb, Whatweb (en su modo agresivo, con `-aggression=4`, aunque tarda mucho), etc.



Episodio 3 – El centinela de Hawkins: Suricata contra el UpsideDown (3.A y 3.B)

Después del caos del Episodio 2, en el que el grupo usó Snort para detectar señales simples del UpsideDown (pings anómalos, accesos prohibidos, criaturas explorando rutas), el Dr. Owens convoca a los estudiantes a una nueva reunión en el Laboratorio Nacional de Hawkins. Eleven ha notado que algunas criaturas del UpsideDown están desarrollando patrones de comunicación más complejos, capaces de ocultarse entre tráfico legítimo. Snort ha sido útil, pero el laboratorio necesita ahora un sistema más potente, multiproceso y detallado, capaz de registrar todo en JSON, analizar múltiples protocolos, actuar como IDS pero también como IPS si es necesario. Y aquí aparece el nuevo personaje, un centinela multihilo llamado **Suricata** que es capaz de ver más lejos que Snort, de actuar más rápido, y de analizar el portal desde múltiples dimensiones del tráfico.

Hoy dominarás este nuevo poder.

3.1. Introducción a Suricata

Suricata (<https://suricata.io/>) es un motor IDS/IPS de código abierto desarrollado por la Open Information Security Foundation (OISF, <https://oisf.net/>), capaz de:

- Inspeccionar múltiples protocolos (HTTP, DNS, TLS, SSH, SMB, ...).
- Procesar tráfico en paralelo (multihilo real).
- Utilizar reglas tipo Snort.
- Generar eventos detallados en EVE JSON, que luego pueden verse en herramientas como Evebox o incluso un ELK Stack.

Es uno de los IDS/IPS más empleados hoy en día en múltiples organizaciones y empresas, por lo que conocerlo es fundamental para futuros ingenieros en ciberseguridad.

Aunque Snort 3 permite JSON output y modo IPS con NFQUEUE, en su instalación y configuración real es frágil, inconsistente y depende de módulos y DAQs que no siempre están disponibles. Suricata, en cambio, ofrece de forma nativa y estable el registro en formato EVE JSON, la ejecución multiproceso y el modo IPS mediante NFQUEUE fiable incluso con solo dos máquinas virtuales como va a ser nuestro caso.

Por ello, en este Episodio 3 nos centraremos en las capacidades IDS/IPS y de visualización de Suricata, sin necesidad de profundizar en la sintaxis de reglas (ya que es común con Snort y ya la conocemos).

3.1.1. INSTALACIÓN Y CONFIGURACIÓN DE SURICATA

Para instalar Suricata en nuestra máquina virtual SEED, que será de nuevo nuestra máquina a proteger y hará por tanto de víctima, primero nos aseguraremos de tener la máquina totalmente actualizada (`sudo apt update` && `sudo apt upgrade -y`). Este proceso puede ser muy muy muy lento, por lo que mucha paciencia. Tras este proceso tendremos que instalar Suricata desde el PPA oficial de OISF con

```
sudo apt install suricata -y
```

Y si no encuentra el repositorio habrá que añadirlo con `sudo apt-get install software-properties-common` && `sudo add-apt-repository ppa:oisf/suricata-stable` && `sudo apt-get update`.

Para verificar que la instalación ha sido correcta, compruébalo con `suricata -V` o `suricata --build-info`, o directamente poniendo `suricata` y que te muestre las opciones que tiene.

Suricata tiene un fichero de configuración en `/etc/suricata/suricata.yaml`, que modificaremos ahora para configurarlo correctamente. Se trata de un fichero muy extenso, pero que únicamente nos fijaremos en el valor de la `HOME_NET`, que deberemos modificar para que tenga las direcciones de nuestra red interna (en este caso que únicamente vamos a defender una máquina, será su ip con máscara /32). En caso de querer lanzar Suricata como servicio de sistema también tendría que modificar el valor de la interfaz de red donde lo queremos poner a escuchar. Sin embargo, en esta práctica vamos a lanzar Suricata siempre de forma manual mediante el comando y ahí podremos especificar la interfaz.

La carpeta por defecto de las reglas es `/var/lib/suricata/rules/` y el fichero de reglas es `suricata.rules`, (si no encontramos este fichero en la carpeta, ejecutar `sudo suricata-update --no-check-certificate` para actualizar las reglas). Mientras que para los logs la carpeta por defecto será `/var/log/suricata/` y se almacenarán las alertas en varios formatos (*fast.log*, *eve.json*,...).

3.1.2. COMANDOS Y REGLAS EN SURICATA

Las reglas que emplearemos en Suricata serán las mismas que empleamos anteriormente en Snort, ya que son compatibles. Es cierto que Suricata añade opciones propias a las reglas, como buffers especializados, inspección automática de múltiples protocolos, más operadores o más logs y contexto, pero no vamos a emplearlas en nuestra práctica.

En cuanto al comando `suricata`, emplearemos principalmente las siguientes opciones:

- c <configfile>: especifica el fichero de configuración, normalmente `/etc/suricata/suricata.yaml`.
- S <rulesfile>: carga nuevas reglas desde el archivo `.rules`, sustituyendo (si es “S” mayúscula) o añadiendo (si es “s” minúscula) al que viene por defecto que es `/var/lib/suricata/rules/suricata.rules`.
- l <logfolder>: especifica la carpeta donde se almacenan los logs de alertas, por defecto en `/var/log/suricata/`.
- i <interface>: especifica la interfaz por la que debe escuchar Suricata.
- k none: deshabilita la verificación interna que hace Suricata del checksum.
- q <num>: permite que Suricata reciba paquetes desde la cola <num> de NFQUEUE para funcionar como IPS.

3.2. Detectando y bloqueando señales del UpsideDown

En este apartado, Eleven y el equipo del Laboratorio Nacional de Hawkins han conseguido activar a Suricata, un centinela avanzado capaz no solo de detectar patrones oscuros del UpsideDown... sino también de bloquearlos antes de que crucen la Brecha. Vamos a replicar los experimentos que hicimos con Snort, pero ahora usando toda la potencia de Suricata.

3.2.1. DETECTANDO SEÑALES ICMP SALIENTES DESDE HAWKINS LAB (IDS)

El laboratorio ha detectado que algunas máquinas comprometidas envían Echo Requests (ICMP tipo 8) hacia el exterior, como si buscaran “contacto” con el UpsideDown. Queremos detectar *cuando la víctima envía esos pings*, pero no cuando recibe los pings desde Kali. En `local.rules` añade la misma regla que en Snort:

```
alert icmp $HOME_NET any -> any any (msg:"Ping saliente detectado"; itype:8;  
sid:1000010; rev:1;)
```

Ahora inicia Suricata en la máquina SEED en su interfaz correcta (espera a que ponga “Engine started”)

```
sudo suricata -c /etc/suricata/suricata.yaml -i <interfaz> -S <carpeta_reglas> -l  
<carpeta_logs> -k none
```

Haz un ping para que salte la alerta. Visualiza el fichero de log *fast.log* y comprueba que se está registrando la actividad. Para poder dejar abierto el fichero de logs mientras hacemos pruebas y ver cómo se van registrando los eventos, se puede emplear el comando `tail -f <logfile>`.

3.2.2. BLOQUEANDO LA NAVEGACIÓN A SITIOS PROHIBIDOS (IPS)

Volvemos a intentar, esta vez con Suricata, detectar cuándo desde la máquina víctima se pretende acceder a un sitio prohibido, como *www.wikipedia.org*. En este caso, además, vamos a bloquear el tráfico, poniendo Suricata en modo IPS. Para ello Suricata no puede limitarse a escuchar en una interfaz de red, sino que necesita integrarse con el firewall del sistema operativo mediante **NFQueue**, un mecanismo del kernel de Linux que permite redirigir paquetes a una cola donde un proceso en espacio de usuario, en este caso Suricata, los inspecciona y decide si los acepta o los descarta.

Antes de comenzar, verifica que se puede navegar sin problemas a *www.wikipedia.org* (como hicimos en Snort, es mejor probar con el comando `curl -v` que con un navegador).

El primer paso es configurar las reglas de **iptables** para redirigir el tráfico de la interfaz de salida a Internet a la cola número 1:

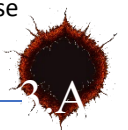
```
sudo iptables -I INPUT -i <interfaz> -j NFQUEUE --queue-num 1 sudo  
iptables -I OUTPUT -o <interfaz> -j NFQUEUE --queue-num 1
```

Ahora emplearemos la regla para bloquear (drop) las solicitudes que salgan de la máquina víctima con el contenido “wikipedia.org” en el campo Host del `http_header`. Para eso podemos emplear un buffer específico de Suricata para este campo, quedando así la regla:

```
drop http any any -> any any (msg:"Acceso prohibido a Wikipedia";  
flow:established,to_server; content:"wikipedia.org"; http_host; sid:1000025; rev:1;)
```

Finalmente, se arranca Suricata en modo IPS indicando que debe escuchar en la cola 1 mediante la opción `-q`, sustituyendo la opción `-i <interfaz>`, que ahora no hace falta (el tráfico está ya especificado en las reglas `iptables`).

Generamos tráfico hacia *www.wikipedia.org* y, si todo ha ido bien, se quedará bloqueado el comando y se registrará la actividad en los logs. Visualiza y comprueba los mismos.



3.2.3. DETECTANDO ATAQUES DE RECONOCIMIENTO CON SURICATA

Volvemos ahora a probar los mismos cuatro ataques de reconocimiento que desde la máquina Kali hicimos a nuestra pobre máquina SEED víctima con Snort (apartado 2.4), pero esta vez con Suricata.

Antes de continuar, quitaremos las reglas `iptables` para volver al modo IDS. Para ello basta con poner `-D` en lugar de `-I` en las mismas:

```
sudo iptables -D INPUT -i <interfaz> -j NFQUEUE --queue-num 1 sudo
iptables -D OUTPUT -o <interfaz> -j NFQUEUE --queue-num 1
```

Recuerda también iniciar un servidor web en la máquina SEED, pues muchos de los ataques buscarán actuar sobre un servicio web.

Cuando ejecutemos ahora Suricata tendremos que tener cuidado al elegir la interfaz, pues ahora los ataques provienen de la máquina Kali, y además tenemos que recordar quitar las reglas propias y dejar las de por defecto (paciencia, pues este conjunto de reglas tardará más en cargarse en el Suricata).

Visualiza y analiza los logs y las alertas activadas. Compara con las activaciones de las reglas de la comunidad Snort.

3.3. Visualizando las señales del UpsideDown

Cada vez que Suricata detecta algo extraño —un Eco Request sospechoso, un acceso prohibido a Wikipedia, un intento de enumeración del UpsideDown— deja registrada una marca en su diario de visiones: el fichero **eve.json**. Este fichero contiene todas las huellas que el Centinela detecta: alertas, flujo de conexiones, eventos HTTP, DNS, TLS, metadatos del tráfico, información de IPS (drop / allowed)...

Sin embargo, leer un fichero JSON gigante a ojo no es tarea sencilla, ni siquiera para Eleven. Por eso en este apartado aprenderemos a visualizarlo con herramientas apropiadas.

Suricata guarda todos sus registros por defecto en la carpeta `/var/log/suricata/` en diversos formatos. Uno de ellos es el fichero JSON `eve.json`. En él, cada línea es un evento independiente en formato JSON. Por ejemplo:

```
1 {
2   "timestamp":"2026-03-17T10:23:12.123Z",
3   "event_type":"alert",
4   "src_ip":"10.0.2.13",
5   "dest_ip":"10.0.2.14",
6   "alert":{
7     "signature_id":300002,
8     "signature":"[UpsideDown] BLOQUEADO: acceso prohibido a Wikipedia",
9     "category":"Attempted Information Leak"
10  },
11  "action":"blocked"
12 }
```

El visor más ligero y recomendado para nuestro laboratorio es el `jq`, un procesador de JSON muy ligero y potente, que podemos instalar directamente con `sudo apt install jq -y`.

Podríamos leer todo el fichero con `jq . /var/log/suricata/eve.json`, pero lo interesante realmente es poder filtrar su información, por ejemplo mostrando únicamente las alertas:

```
jq 'select(.event_type=="alert")' /var/log/suricata/eve.json
```

o ver solo las alertas con DROP con:

```
jq 'select(.alert!=null and .action=="blocked")' /var/log/suricata/eve.json
```

o ver los ataques más frecuentes mostrando las firmas activadas ordenadas:

```
jq -r 'select(.alert).alert.signature' /var/log/suricata/eve.json | sort | uniq -c |  
sort -rn | head -20
```

o mostrar solo los eventos HTTP

```
jq 'select(.event_type=="http")' /var/log/suricata/eve.json
```

o el origen y destino de los ataques:

```
jq -r 'select(.event_type=="alert") | [.src_ip, .dest_ip] | @csv'  
/var/log/suricata/eve.json
```

o hacer un análisis temporal visualizando el timestamp y el mensaje (para ello recordad la importancia de tener los relojes en hora y sincronizados en nuestra red):

```
jq -r 'select(.alert) | [.timestamp, .alert.signature] | @tsv'  
/var/log/suricata/eve.json
```

Suricata puede integrarse con visores más avanzados como **Evebox**, **Splunk Free** o **ELK**, pero estos consumen muchos recursos y no siempre funcionan en la máquina virtual SEED. Por eso, en este laboratorio, hemos usado *jq* como visor principal, y animamos a los/as estudiantes más curiosos a intentar instalar Evebox o Splunk como ejercicio opcional para comprender cómo se visualizarían los eventos en un sistema de monitorización real.

Por ejemplo, Splunk se podría instalar fácilmente con:

```
sudo wget -O splunk.deb 'https://download.splunk.com/products/splunk/releases/9.0.0/  
linux/splunk-9.0.0-6818ac46f2ec-linux-2.6-amd64.deb' --no-check-certificate sudo  
dpkg -i splunk.deb sudo /opt/splunk/bin/splunk start --accept-license
```

Y visualizar el portal web de Splunk en <http://localhost:8000>. Una vez dentro siguiendo los menús *Settings -> Data Inputs -> Files & Directories -> New*, se puede poner la ruta del fichero *eve.json*, e ir pasando a los siguientes pasos. Podremos entonces hacer búsquedas como *index=* event_type=alert*. Y cuando queramos cerrar Splunk debemos emplear el comando `sudo /opt/splunk/bin/splunk stop`.



Aquí termina el episodio y la temporada 3. Con los sensores en funcionamiento, las firmas ajustadas y cada alerta correctamente interpretada, el Hawkins National Laboratory consigue bloquear el último eco procedente del Upside Down. El tráfico anómalo se desvanece, las rutas sospechosas quedan selladas y las máquinas vuelven a latir en silencio. El Escuadrón Hawkins confirma lo que llevaba horas intentando lograr: el portal ha sido cerrado. La intrusión ha terminado.

Pero aunque la red queda limpia y estable, los científicos saben que la contención no es el final. Si un enemigo es capaz de infiltrarse, también puede intentar escuchar, interceptar o manipular las comunicaciones que unen las distintas zonas del laboratorio. La siguiente batalla ya empieza a vislumbrarse: proteger no solo lo que ocurre dentro de la red, sino cómo viaja la información entre sus nodos.

Pronto llegará la Temporada 4: “Túneles VPN – IPsec y TLS en la brecha”, donde el Escuadrón Hawkins aprenderá a blindar cada conexión, encapsular cada mensaje y asegurar que ningún rastro del Upside Down pueda ocultarse en los propios canales de comunicación.



SEGURIDAD EN REDES

FIN DE LA TEMPORADA 3

**EL PORTAL HA SIDO CERRADO.
LA INTRUSIÓN HA TERMINADO.**



**GRADO EN INGENIERÍA DE LA CIBERSEGURIDAD
DIRECTED BY CÉSAR CÁCERES TALADRIZ**